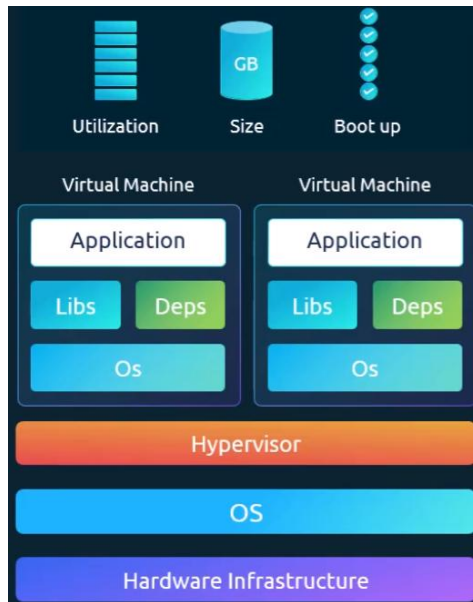
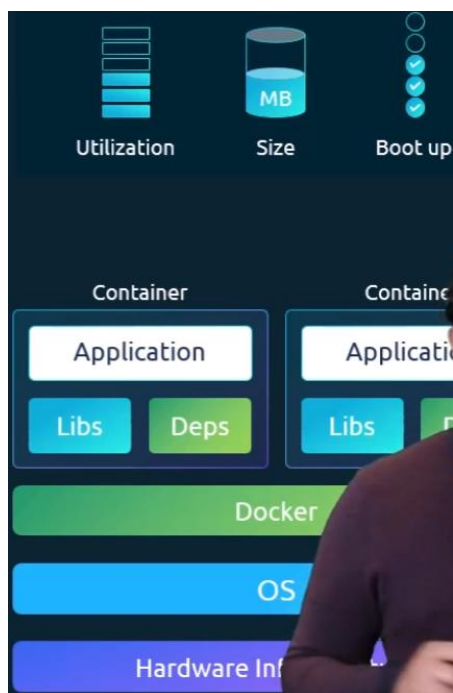


- Docker containers share the same Kernel of the host machine.
 - ⌘ If the system has linux kernel, then you can't run windows container in that system.
 - ⌘ The OS of the container should be compatible with the kernel of the host machine.



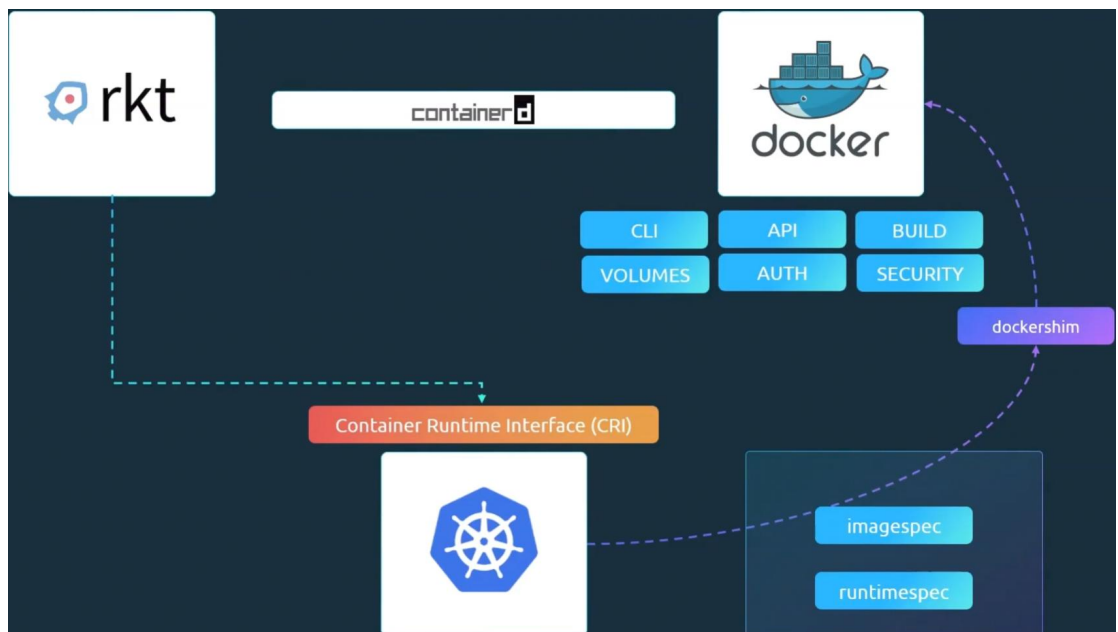
(virtual machine)



(Docker)

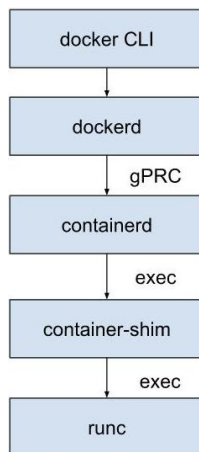
- When you install Kubernetes in a system, then the following things are installed:
 - ⌘ API Server
 - ⌘ It acts as the frontend of kubernetes cluster.
 - ⌘ etcd
 - ⌘ Distributed key-value store that keeps the cluster information(i.e. all details about worker nodes and master nodes) of kubernetes to manage the cluster.
 - ⌘ Kubelet
 - ⌘ Make sure containers are running inside the nodes.
 - ⌘ Container Runtime
 - ⌘ Underlying software used to run software.
 - ⌘ Docker has “containerd”.
 - ⌘ Controller
 - ⌘ Brain behind the orchestration.
 - ⌘ It takes decision to bring up new containers if nodes or pods goes down.
 - ⌘ Scheduler
 - ⌘ Implementing locks within the cluster to ensure there are no conflicts.
 - ⌘ Assign the nodes to the pods.
- How does nodes are divided into master and slave?
 - ⌘ Each worker node contains *Container Runtime* like containerd. Other container runtime are also available like *rkt*, *CRI-O*.
 - ⌘ Master node contains **kube-apiserver** and Worker node contains **kubelet** , this kube-apiserver makes the node master.
 - ⌘ **Kubelet** agent of the worker node connects with the master node to do the things.
- Basic commands:
 - ⌘ **kubectrl run hello-minikube**
kubectrl run command is used to deploy an application to the cluster.
 - ⌘ **kubectrl cluster-info**
it gives the information about the **cluster**.
 - ⌘ **kubectrl get nodes**
it list all the nodes of the cluster.
- Docker vs containerd
 - ⌘ Kubernetes introduced an interface called CRI (container runtime interface)
 - ⌘ CRI allowed every vendor to work as a container runtime for kubernetes as long as they follow the OCI (open container initiative) standard.

- ⌘ This CRI has 2 specs:
 - ⌘ imagespec
 - it is the specification about how an image should be built.
 - ⌘ runtimespec
 - it is the specification about how any container runtime should be developed.
- ⌘ Docker was there way before CRI was introduced. So, Kubernetes introduced ***dockershim*** to use Docker which was a temporary solution.
- ⌘ Now, *Docker Runtime* (containerd) contains inbuilt CRI.



- ⌘ Docker has multiple things that work like docker cli, api, volumes etc etc.
- ⌘ **containerd** is CRI compatible, can be used as a container runtime separately with Kubernetes.
- ⌘

➤ **NOTE about Docker architecture layers** -----



➤ When you run **docker run nginx**, internally the below happens

- ♣ **Docker CLI** sends request to **dockerd**.
- ♣ **dockerd** (*docker daemon*) does image pull, container setup, volume setup, container config.

dockerd is the brain of docker.

Then, it calls **containerd**.

- ♣ **containerd** calls **runc**.

It actually creates a container (namespace, cgroups)

➤ **docker = dockerd (manager) + containerd (executor) + runc (low-level runner)**

➤ The parts of docker:

- ♣ Docker CLI
 - ✧ the **docker** command you type.
 - ✧ It talks to the docker daemon (**dockerd**) via REST API.
- ♣ Docker Daemon (**dockerd**)
 - ✧ Its brain of Docker.
 - ✧ It handles containers, images, networks, volumes.
 - ✧ Exposes API for CLI/Tools.
- ♣ Docker API
 - ✧ REST API used by CLI, or external tools.
 - ✧ Ex: Docker Desktop, CI tools
- ♣ Docker Build System
 - ✧ Builds images from *Dockerfile*.
 - ✧ Internally uses **Buildkit** (modern Docker).
- ♣ Docker objects (managed by Docker)
 - ✧ Containers, volumes, networks, images.

➤ The parts outside of Docker.

⌘ **Container Runtime** (high level)

- ⌘ containerd, it manages pulling images, container lifecycle.
- ⌘ Originally part of docker, now a separate CNCF project.

⌘ **Low Level Runtime**

- ⌘ **runc**
- ⌘ Actually creates containers using namespace, cgroups.

➤ How images are built?

- ⌘ Docker is something that calls **containerd** which further calls **runc** to create an image.

- ⌘ Any vendor can use **containerd** and create image.

- ⌘ It doesn't mean that docker is just a tool like others. It's a complete platform which contains

- ⌘ CLI (docker)
- ⌘ Daemon (dockerd)
- ⌘ Image build system (buildkit)
- ⌘ Networking, Volumes, Image management

- ⌘ It internally uses **containerd** and **runc**.

- ⌘ **containerd**: pull, start, stop the container

runc: actually runs the container (namespaces, cgroups)

- ⌘ Before

kubernetes -- dockershim -- Docker -- containerd -- runc

- ⌘ Now

kubernetes -- containerd -- runc

- ⌘ Kubernetes removed docker completely from its pipeline.

- ⌘ You just build the image and push in the registry, rest of the things will be done by Kubernetes.

Because anyways it uses **containerd** which can pull and delegate the call to **runc** to run the container.

Docker now adapted to CRI standard, so it builds CRI standard images that can be directly used by Kubernetes.

- ⌘ Docker is having Docker Networks, Volumes, Lifecycle, **How will they all be managed without Docker in Kubernetes?**

- ⌘ For networking, Kubernetes has **CNI** which is very powerful.

- ✧ For Storage, PersistentVolumes, PersistentVolumeClaims, StorageClasses are there.
- ✧ For lifecycle, pods and replica management is there.
- Kubernetes has **crictl** to talk to containerd in runtime for debugging purpose.
 - ✧ Docker doesn't use *crictl* to talk to containerd (don't get confused on this)
 - ✧ Both **dockerd** and **crictl** talks to *containerd* but in different ways.
 - ✧ *dockerd* talks to *containerd* via its own API (gRPC)
 - ✧ *crictl* talks to *containerd* via CRI plugin (*CRI API*)

```
docker CLI = developer tool (build + run)
crictl      = debugging tool (inspect runtime)
```

docker cli	crictl	Description	Unsupported Features
attach	attach	Attach to a running container	--detach-keys , --sig-proxy
exec	exec	Run a command in a running container	--privileged , --user , --detach-keys
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	--details
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

- ✧ **crictl** can connect to any *CRI* configured runtime, not only *containerd*.
- F

➤ Minikube

- ♣ It provides a single node environment where all the processes of both control plane and data plane are present in a single node.



- ♣ it provides an iso image directly that can be run using any virtual box like VM ware, virtualbox ..etc.
- ♣ **minikube start --driver=virtualbox** (use --driver to run minikube inside virtual machine)

➤ Pods

- ♣ The Nodes doesn't run the application directly. Rather the container is wrapped with an object called as **pod**.
- ♣ It is the smallest object that can be created in Kubernetes.
- ♣ Do not add multiple containers inside a single pod if you want to scale, rather create different pods for that.
And for CPU or other usage, then run the pods in multiple nodes.
- ♣ Its not a restriction that we can create only one container inside a pod, we can create many.

But, those containers **shouldn't be of same kind**.

There are use-cases that create helper containers along with the application container inside the same pods which lives and dies along with the main application container.

- ♣ The application and helper containers can communicate with each other.

➤ Why Helper containers?

- ♣ Lets see only docker containers for now.
- ♣ You have some application containers. But as your app grows, you need to process some data.
So, you created some other containers to process the respective containers.
- ♣ Now, you need to keep shared volumes for each application and helper container pairs.

```

docker run python-app
docker run python-app
docker run python-app
docker run python-app
docker run helper --link app1
docker run helper --link app2
docker run helper --link app3
docker run helper --link app4

```

- ⌘ (-link is the legacy method to link one container to another in docker)

Now, you need to keep track of the containers like

App	Helper	Volume
Python1	App1	Vol1
Python2	App2	Vol2

- ⌘ With **Pods**, Kubernetes does all these by default.

All the containers inside a Pod can access each other, can access the same volume, and created and destroyed together.

- ⌘ Multiple containers are very rare usecase.

➤ To create a **Pod**

```

C:\Users\alokr>kubectl run nginx --image=nginx
pod/nginx created

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS             RESTARTS   AGE
nginx     0/1     ContainerCreating   0           6s

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS             RESTARTS   AGE
nginx     0/1     ContainerCreating   0           13s

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           27s

```

- ⌘ **kubectl run <name> --image=<image name>**

it'll pull the image from docker registry and create a pod and run the container inside that.

5

4

 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F
 F

➤ **F**

➤ **F**

➤ F

➤ F

Commands

➤ **kubectl get nodes -o wide**

```
controlplane ~ ➔ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION          INTERNAL-IP    EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION    CONTAINER-RUNTIME
controlplane   Ready     control-plane   30m   v1.35.0+k3s3     192.168.18.136 <none>         Alpine Linux v3.16 6.8.0-1047-gcp   containerd://1.6.8

controlplane ~ ➔ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
controlplane   Ready     control-plane   30m   v1.35.0+k3s3
```

it gives extra details about the nodes.

⚡ You can see the OS-IMAGE using

cat /etc/os-release

but, for this you need to have access to that node or you need to do ssh to that node and do this.

⚡ On the other hand, **kubecttl** command has all the nodes registered and without ssh or something else it can provide you the details.

➤

➤ F

➤ F

➤ F

➤